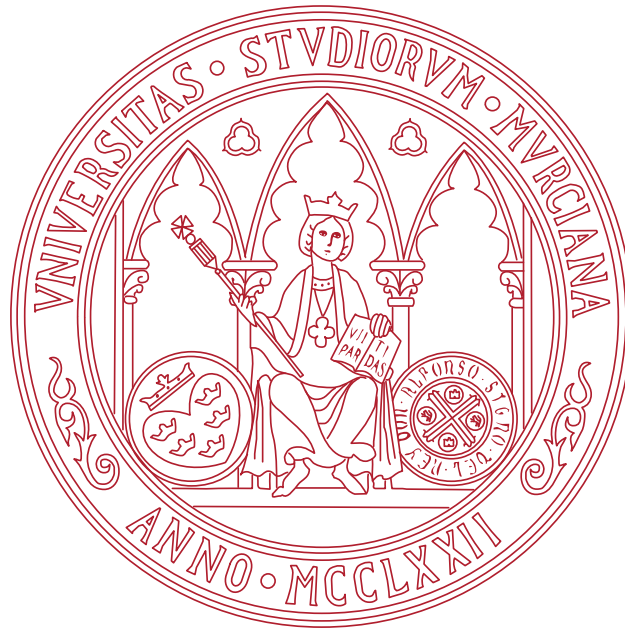




UNIVERSIDAD DE MURCIA

TRABAJO FIN DE GRADO

Impacto de los sistemas de memoria en grandes modelos de lenguaje

Estudiante:

Javier Patricio LUJÁN ROMERO
jp.lujanromero@um.es

Tutor:

Eduardo MARTÍNEZ GRACIÁ
edumart@um.es

Enero 2026

A mi familia por su apoyo incondicional desde el inicio.

Mi madre y su ternura.

Mi padre y su sacrificio.

Índice general

Resumen	3
Extended abstract	4
1. Introducción	5
1.1. Motivación	5
1.2. Objetivos	6
1.3. Sistemas de memoria en grandes modelos de lenguaje	6
1.3.1. Mem0	7
2. Estado del arte	9
3. Análisis de objetivos y metodología	10
4. Diseño y resolución	11
5. Conclusiones y vías futuras	12
Bibliografía	13

Índice de figuras

1.1. Pipeline de Mem0. Extraído de [1].	8
---	---

Resumen

En este proyecto se plantea el problema de...

Extended abstract

This project faces the problem of...

Introducción

1.1. Motivación

En los últimos años la IA, en particular los grandes modelos de lenguaje – LLMs, por sus siglas en inglés –, han supuesto una revolución y un cambio de paradigma ya no solo dentro de la industria del Machine Learning, sino en todos los ámbitos de la sociedad. Las aplicaciones de chat basadas en estos grandes modelos de texto, como ChatGPT o Google Gemini, se han convertido en herramientas principales para millones de usuarios en todo el mundo realizando una inmensidad de tareas distintas.

Estas tareas tienen diferentes niveles de complejidad, desde tareas simples que se resuelven con una respuesta directa, a tareas más complejas en las que el propio modelo debe replantear los conceptos con los que está trabajando y como los ha de usar para llegar a la solución.

Dentro de estas tareas complejas, se pueden plantear dos situaciones: Una en la que el modelo dispone de toda la información necesaria para resolver la tarea en el prompt, y la complejidad radica en el proceso deductivo que el modelo debe realizar para realizar exitosamente la tarea, como por ejemplo la resolución de problemas matemáticos avanzados. Y, por otro lado, tenemos el caso en el que la resolución exitosa de la tarea requiere de otra información adicional que no se encuentra en el prompt, como ejemplo de este tipo de tarea tenemos la modificación de un código fuente ajustándose a los estándares del proyecto en el que se encuentra.

Es dentro de este segundo tipo de tareas donde ha proliferado el uso de los conocidos agentes de IA: sistemas autónomos capaces de percibir su entorno, razonar sobre posibles cursos de acción, y tomar y ejecutar decisiones [2]. Estos agentes tienen como base un LLM principal el cual realiza el razonamiento, y se le dota de un conjunto diverso de herramientas que le permiten interactuar con su entorno. Ejemplos de estas herramientas son: navegadores web, RAG en bases de datos, editores de código, e incluso otros modelos de IA especializados en diferentes tareas.

Pero para el desarrollo de este trabajo, nos vamos a centrar en un tipo de herramienta en concreto: los sistemas de memoria. Estos sistemas permiten a los agentes almacenar y recuperar información relevante a lo largo del tiempo, mejorando su capacidad para manejar tareas que requieren conocimiento a largo plazo y reduciendo la necesidad de incluir grandes cantidades de texto en el prompt. De esta manera se construyen agentes más eficientes tanto a nivel computacional como en el uso de tokens, que por ende son más económicos y generan un menor impacto ambiental.

1.2. Objetivos

El objetivo principal del trabajo es realizar un estudio sobre el impacto que tienen los sistemas de memoria en el rendimiento de los LLMs en distintos términos: latencia, accuracy y consumo de tokens.

Con el fin de realizar este primer objetivo se ha creado una pipeline modular, que permite integrar diferentes sistemas de memoria para LLMs, facilitando la experimentación y evaluación de estos sistemas en diversas tareas y datasets. Cumpliendo de esta manera el segundo objetivo, diseñar un sistema accesible que permita evaluar diferentes sistemas de memoria en LLMs de manera sencilla y adaptable.

1.3. Sistemas de memoria en grandes modelos de lenguaje

Parrafo introductorio de los LLMs.

Los grandes modelos de lenguaje (LLMs) procesan el texto que toman como entrada, dividiendo las palabras en lo que se conoce como tokens. Estos tokens son los que se usan para predecir las siguientes palabras a usar. Sin embargo, el número de tokens que un modelo puede usar es siempre limitado, a este límite es lo que se le conoce como la ventana de contexto. Hay modelos donde la ventana de contexto es más modesta, 32K tokens en qwen2.5, y otros donde la ventana es muy superior, 400K tokens en GPT-5.

(Hacer una intro a que tienen problemas):

1. Un primer problema que tiene la ventana de contexto es que, como indica su propia definición, es limitada. Además esta limitación es mayor cuanto más económico es un modelo, pues modelos más grandes y más computacionalmente costosos suelen tener ventanas de contexto mayores.
2. Otro problema, es que, aunque un texto quepa dentro de la ventana de contexto, se ha probado en diferentes estudios que los modelos pierden la *atención* mientras procesan el texto, dando resultados no deseables aun en franjas en un principio computables (alguna referencia a un estudio).
3. Finalmente, el último desafío que se presenta, es en la propia naturaleza del aprendizaje, la ventana de contexto permite retener información con la que trabajar,

similar a la memoria de trabajo humana, pero es incapaz de procesar la información a lo largo del tiempo, e incluso de diferentes conversaciones y chats.

Como solución a estos problemas, se ha propuesto diferentes enfoques, entre los que destacan los sistemas de memoria, cuyo objetivo es imitar la memoria a largo plazo, permitiendo a los modelos *recordar* la información importante en el momento adecuado. Reduciendo así la necesidad de manejar grandes cantidades de texto. Economizando, e incluso mejorando, el rendimiento de los modelos en tareas que requieren trabajar con conocimiento a largo plazo.

Para que estos sistemas de memoria sean efectivos, se busca que cumplan ciertas características, entre las que destacan:

1. Selección adecuada de la información relevante para almacenar en memoria.
2. Consolidación efectiva de la información almacenada.
3. Recuperación eficiente de la información cuando sea necesario.
4. Capacidad de adaptación y actualización de la memoria con nueva información.

1.3.1. Mem0

Entre los diferentes sistemas de memoria que hay en el mercado, Mem0 ha sido elegido como base para este trabajo, permitiendo realizar la comparativa entre la ventana de contexto y el uso de un sistema de memoria. Esta decisión se ha tomado así por ser Mem0 un ejemplo canónico de sistema de memoria, por su facilidad de uso e integración, y por ser un sistema actual y actualizado.

Es por ende necesario entender el funcionamiento de Mem0 para poder comprender la relación entre la mejora que supone y los posibles factores negativos que se pueden asociar a este.

El componente principal de Mem0 es una base de datos vectorial en la que se almacenarán las memorias conforme se vayan generando. Mem0 actúa de wrapper sobre esta base de datos, permitiendo usar cualquier tipo de bdd vectorial que se desee, lo que aporta una gran flexibilidad y, en caso de que sea necesario, privacidad.

Su comportamiento se divide en dos fases principales: la extracción de posibles memorias de la conversación, y la actualización de la base de datos:

La **extracción** como se aprecia en la figura 1.1, se realiza en cada iteración del chat, tomando como entrada el último par de mensajes –usuario y respuesta del modelo–; una secuencia con los últimos m mensajes, siendo este número un hiperparámetro, y finalmente un resumen de la conversación generado de manera asincrónica en la base de datos. Toda esta información se introduce en un prompt y se envía a un LLM, previamente configurado por el operador del sistema, con el fin de que este extraiga memorias candidatas para añadir a la base de datos.

Una vez realizado este proceso se procede con la **actualización**. En esta fase se realiza

el mismo procedimiento para cada memoria candidata extraída en el paso anterior que consiste en realizar una búsqueda semántica en la base de datos tomando como prompt la memoria candidata. Entonces, usando tanto la memoria candidata como las memorias obtenidas en la consulta, se realiza una segunda llamada a un LLM el cual debe decidir entre cuatro posibles acciones: *ADD*, *UPDATE*, *DELETE* y *NOOP*. Que significan:

- **NOOP**: No hacer nada, la memoria candidata no aporta nada nuevo a la base de datos.
- **UPDATE**: Actualizar alguna de las memorias obtenidas en la consulta con la información de la memoria candidata.
- **ADD**: Añadir la memoria candidata como una nueva memoria en la base de datos.
- **DELETE**: Borrar alguna de las memorias obtenidas en la consulta, pues ya no es relevante.

En la figura 1.1 se puede observar toda la pipeline de Mem0.

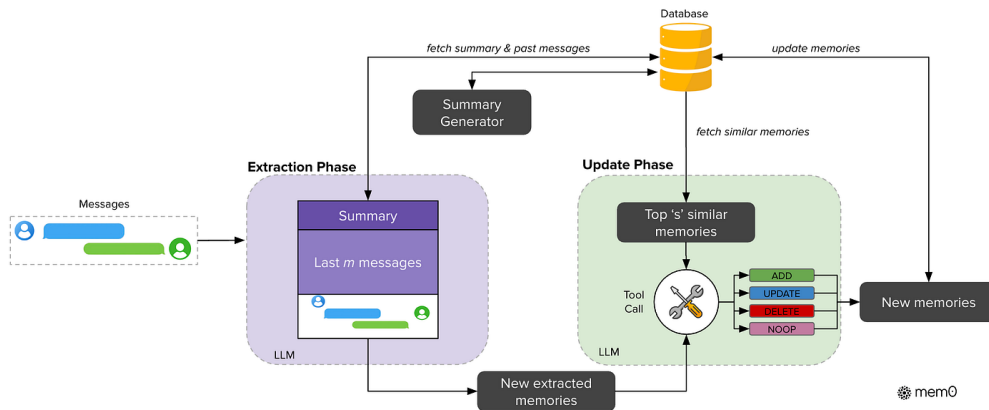


Figura 1.1: Pipeline de Mem0. Extraído de [1].

CAPÍTULO 2

Estado del arte

En este capítulo se realiza un estudio del estado del arte relativo al objetivo de este proyecto.

CAPÍTULO 3

Análisis de objetivos y metodología

Una vez tratado el estado del arte, realizamos en este capítulo el análisis de los objetivos del mismo, estableciendo los elementos concretos que nos van a permitir llevar a cabo dichos objetivos así como la metodología de desarrollo.

CAPÍTULO 4

Diseño y resolución

En este capítulo se realiza la descripción de la implementación de...

CAPÍTULO 5

Conclusiones y vías futuras

Finalizamos el trabajo estableciendo las conclusiones y vías futuras...

Bibliografía

- [1] Prateek Chhikara, Dev Khant, Saket Aryan, Taranjeet Singh, and Deshraj Yadav. Mem0: Building production-ready ai agents with scalable long-term memory. *arXiv preprint arXiv:2504.19413*, 2025.
- [2] Xiaodong Qu, Andrews Damoah, Joshua Sherwood, Peiyan Liu, Christian Shun Jin, Lulu Chen, Minjie Shen, Nawwaf Aleisa, Zeyuan Hou, Chenyu Zhang, et al. A comprehensive review of ai agents: Transforming possibilities in technology and beyond. *arXiv preprint arXiv:2508.11957*, 2025.